NEAT とトポロジー進化・間接符号化

NEAT (2002)は構造と重みを同時に進化させ、歴史的マーキング・種分化・最小構造からの複雑化を確立した。
rtNEAT で実時間化、CPPN/HyperNEAT で座標から結合を生む間接符号化へ、DPPN・CoDeepNEAT・WANN で深層学習と接続。
2024 ～ 26 年は TensorNEAT による GPU 化と四分木の限界、低ランク分解型の符号化、FER 仮説による表現の見直しへ。

#	年	論文	発表	系譜
1	2002	NEAT (Stanley & Miikkulainen 2002)	Evolutionary Computation 10(2) 2002	原典
2	2005	rtNEAT / NERO (Stanley+ 2005)	IEEE TEC 9(6) 2005	発展
3	2007	HyperNEAT (Gauci & Stanley 2007)	GECCO 2007	発展
4	2016	DPPN (Fernando+ 2016)	GECCO 2016	発展
5	2017	CoDeepNEAT (Miikkulainen+ 2017)	arXiv プレプリント 2017 (書籍章 2019)	発展
6	2019	WANN (Gaier & Ha 2019)	NeurIPS 2019 (spotlight)	発展
7	2024	TensorNEAT (Wang+ 2024)	GECCO 2024 / ACM TELO 2025	最新
8	2025	低ランク分解による間接符号化 (Garbus & Pollack 2025)	arXiv プレプリント 2025	最新
9	2025	FER 仮説 (Kumar+ 2025)	arXiv プレプリント 2025 (position paper)	最新
10	2026	Quadtree Bottleneck / JAX-ESHN (Claret+ 2026)	arXiv プレプリント 2026	最新

Evolving Neural Networks through Augmenting Topologies

Kenneth O. Stanley, Risto Miikkulainen（The University of Texas at Austin）| Evolutionary Computation 2002

1 どんなもの？

重みとトポロジーを同時に進化させる NEAT を提案。①歴史的マーキングによる交叉 ②種分化による革新の保護 ③最小構造からの漸進的な複雑化、の 3 要素で、速度なし二重倒立振子を ESP の 1/5 、Cellular Encoding の 1/25 の評価回数で解いた。

2 先行研究と比べてどこがすごい？

固定トポロジーの ESP が構造進化の Cellular Encoding より速く、構造を進化させる意義は疑問視されていた。異なる構造間の交叉（competing conventions）、新構造の早期淘汰、ランダム初期化による肥大という TWEANN の 3 課題を、トポロジー解析なしで同時に解いた。

3 技術や手法のキモはどこ？

新しい遺伝子に通し番号（innovation number）を付け、同番号を相同とみなして整列・交叉する。番号の不一致から距離 $\delta = c1 \cdot E/N + c2 \cdot D/N + c3 \cdot \bar{W}$ を測り、閾値で種に分けて適応度を共有。全個体を隠れ層なしで始め、ノード追加・結合追加の変異で必要な分だけ成長させる。

4 どうやって有効だと検証した？

XOR は 100 試行すべて成功（平均 32 世代、隠れ 2.35 個）。速度あり DPV で 3,600 評価（ESP 3,800 、SANE 12,600 : Table 1 ）、速度なし DPNV で 33,184 評価（ESP 169,466 、CE 840,000 : Table 2 ）。成長なし 30,239（失敗 80% ）など、どの要素を外しても悪化（Table 3 ）。

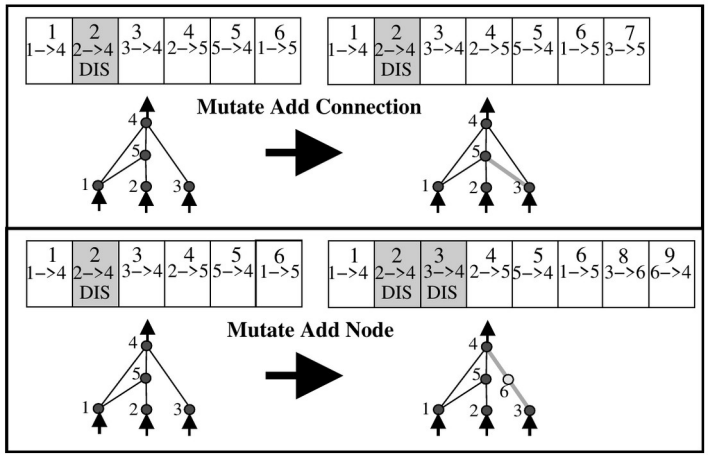
5 議論はある？

著者：重要なのは最終構造ではなく途中解の次元を小さく保つこと。競争的共進化で戦略を複雑化し続けられると期待。

所感：ベンチマークは倒立振子中心で小規模。直接符号化なので大規模ネットには向かず、そこを HyperNEAT が補う。

6 次に読むべき論文は？

- rtNEAT (2005)（本資料）
- HyperNEAT (2007)（本資料）
- 競争的共進化と複雑化 (Stanley+ 2004)（未調査）
- Novelty Search (2011)（既存：open-endedness/quality-diversity ）



出典：NEAT Fig. 3（結合追加・ノード追加の 2 種類の構造変異と innovation number ）

書誌情報

発表： Evolutionary Computation 10(2) 2002
リンク：
https://doi.org/10.1162/106365602320169811
系譜：原典
調査日： 2026-10-27
確認範囲：全文（著者公開 PDF ：問題提起・手法・実験・アブレーション・考察）

Real-Time Neuroevolution in the NERO Video Game

Kenneth O. Stanley, Bobby D. Bryant, Risto Miikkulainen（UT Austin）| IEEE TEC 2005

1 どんなもの？

NEAT を世代交代なしで連続的に動かす rtNEAT を提案し、プレイヤーが訓練課題を設計してロボット部隊を育てる機械学習ゲーム NERO を構築。ゲーム中の数十秒〜数分で行動が目に見えて改善することを示した。

2 先行研究と比べてどこがすごい？

ゲーム AI はスクリプト固定か出荷前のオフライン学習で、プレイ中に適応しなかった。Q 学習などは全行動の価値計算、ε 探索による不自然な挙動、個体の均質化が問題。先行の実時間進化（Agogino+ 2000）は固定トポロジーで、複雑化も種分化もなかった。

3 技術や手法のキモはどこ？

数 tick ごとに、最低生存時間 m を超えた個体のうち調整後適応度が最小のものを除き、種の平均適応度に比例した確率で親種を選んで子を 1 体だけ作る。閾値 δt を動的に調整して再種分け。不適格割合 $I=m/(|P|\cdot n)$ から置換間隔 n を決め、NEAT の種分化動態を近似する。

4 どうやって有効だと検証した？

50 体、Torque エンジンで検証。敵への接近は平均 99.7 秒で 9 割の個体が習得（10 試行）。迷路走破や砲台の背後からの攻撃も獲得し、種ごとに別経路を選んだ。対戦では接近型が回避型に 6 勝 3 敗 1 分、壁際戦術チームは接近型に 10 戦全勝（Table 1, 2）。

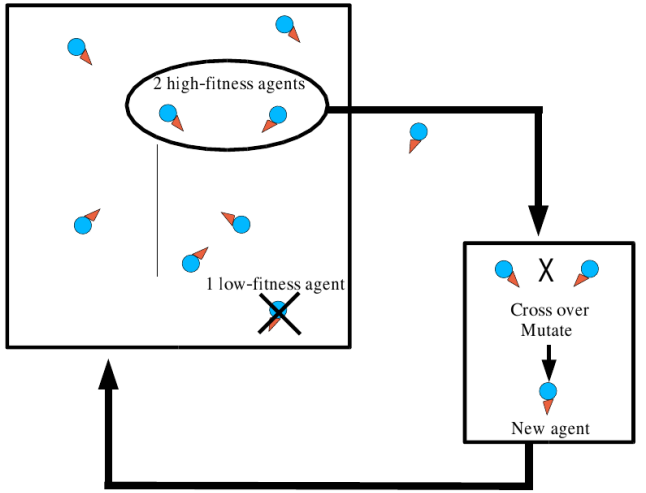
5 議論はある？

著者：評価が主観的で、基準チームとの対戦による定量化が課題。教育・訓練への応用を展望。

所感：成果の多くはスクリーンショットと対戦結果による定性的評価。照準はスクリプト補助で、学習が担う範囲は限られる。

6 次に読むべき論文は？

- NEAT (2002)（本資料）
- Picbreeder (Secretan+ 2011)（未調査）
- odNEAT (Silva+ 2012)（未調査）
- Continual RL with Neuroevolution (2026)（未調査）



出典：rtNEAT Fig. 5（高適応度 2 体の子で低適応度 1 体を置き換える連続置換サイクル）

書誌情報

発表： IEEE TEC 9(6) 2005
リンク：
https://doi.org/10.1109/TEVC.2005.856210
系譜：発展（← NEAT）
調査日： 2026-10-27
確認範囲：全文（著者公開版 PDF：問題提起・手法・NERO 実験・考察）

Generating Large-Scale Neural Networks Through Discovering Geometric Regularities

Jason Gauci, Kenneth O. Stanley（University of Central Florida）| GECCO 2007

1 どんなもの？

NEAT で CPPN（座標を入力に値を返す合成関数ネット）を進化させ、その出力を基板上の全ニューロン対の結合重みとして読む HyperNEAT を提案。11×11 で進化させた解を、再学習なしで 55×55（約 900 万結合）に拡大しても性能を保った。

2 先行研究と比べてどこがすごい？

NEAT のような直接符号化は結合を 1 本ずつ見つける必要があり、高次元化に弱い。Cellular Encoding など発生を模した間接符号化は成長の模擬が必要だった。CPPN（Stanley 2007）は発生を省いて対称・反復を表せるが、画像生成に使われるのみで、結合の符号化には使われていなかった。

3 技術や手法のキモはどこ？

基板上のノードに座標を与え、CPPN(x1,y1,x2,y2)=w で全結合を問い合わせ、|w|≤0.2 なら結合なしとする。CPPN はガウス（対称）や sin（反復）を合成するため、結合に局所性・対称・反復が自然に現れる。同じ CPPN を高解像度の基板に再度問い合わせられる。

4 どうやって有効だと検証した？

大小 2 物体のうち大きい方の中心を当てる視覚課題（75 配置、20 run）。直接符号化の P-NEAT（14,641 次元）より有意に正確で汎化も良い（Fig. 7a）。解の CPPN は平均 24 結合で基板（12,827 結合）の 1/534。33×33・55×55 に拡大しても性能が落ちない（Fig. 7b）。

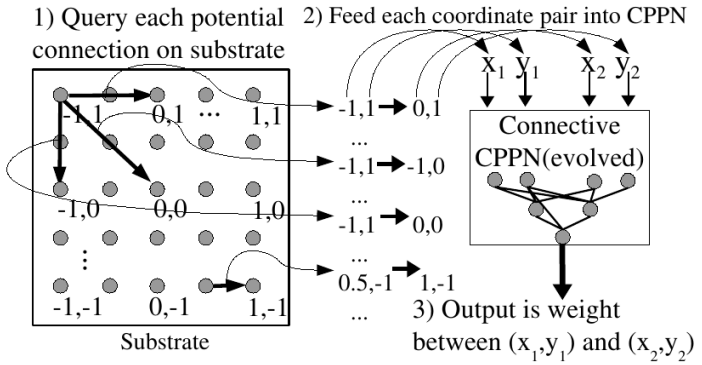
5 議論はある？

著者：低解像度で概念を学び、高解像度で進化を続ける視覚応用などを展望。

所感：幾何的規則性そのものが解になる課題で有利な設定。基板配置は人が決める必要があり、その自動化が ES-HyperNEAT。ALife 2009 の雑誌版は本文未確認。

6 次に読むべき論文は？

- CPPN (Stanley 2007)（本文未確認）
- ES-HyperNEAT (Risi & Stanley 2012)（本文未確認）
- DPPN (2016)（本資料）
- TensorNEAT (2024)（本資料）



出典：HyperNEAT Fig. 3（基板上の各結合を座標対として CPPN に問い合わせ、出力を重みとする）

書誌情報

発表： GECCO 2007
リンク： https://doi.org/10.1145/1276958.1277158
系譜： 発展（← NEAT / CPPN）
調査日： 2026-10-27
確認範囲： 全文（GECCO 2007 会議版：問題提起・手法・実験・考察。雑誌版 ALife 2009 は本文未確認）

Convolution by Evolution: Differentiable Pattern Producing Networks

Chrisantha Fernando, Dylan Banarse, Malcolm Reynolds ほか（Google DeepMind）| GECCO 2016

1 どんなもの？

CPPN のトポロジーは進化させ、重みは誤差逆伝播で学習する微分可能版 DPPN を提案（学習した重みを子に引き継ぐラマルク型）。全結合デノイジング自己符号化器の 157,684 パラメータを約 200 個の DPPN パラメータで生成し、畳み込み状の受容野を再発見した。

2 先行研究と比べてどこがすごい？

CPPN/HyperNEAT は重みも進化で探すため大規模パラメータには非効率で、HyperNEAT で LeNet-5 の重みを進化させても MNIST 正解率は約 50% だった。勾配学習のスケーラビリティと進化による構造探索を組み合わせ、間接符号化そのものを微分可能にした。

3 技術や手法のキモはどこ？

DPPN は座標ベクトル（x, y, 中心からの距離、層番号など）を受け取り、大きなネットの各重みを出力する。適応度評価のたびに Adam で 1000 ステップ学習し、自己符号化器の損失勾配を DPPN の重みまで逆伝播。外側の GA がノード・辺の追加削除と交叉（ネットの併合）を行う。

4 どうやって有効だと検証した？

単一数字の再構成で MSE はラマルク 0.021、ボールドウィン 0.037、ダーウィン 0.079（10 run 平均）。交叉ありで 0.005、なしで 0.021。187 パラメータの DPPN が MNIST デノイジングで BCE 0.091 を出し、同数パラメータの直接符号化（>0.24）を上回り Omniglot にも汎化（Fig. 6, 7）。

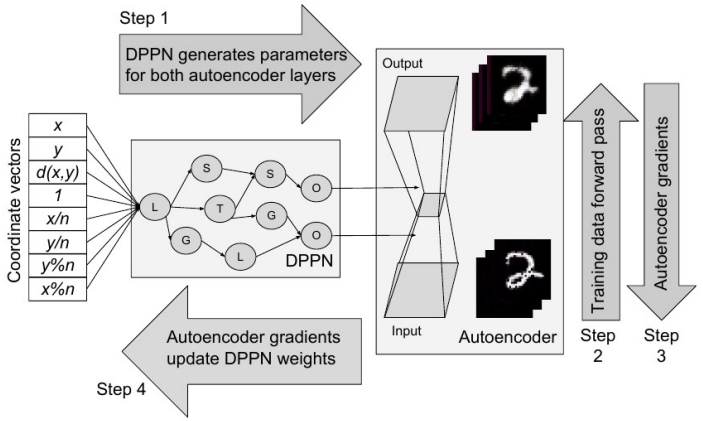
5 議論はある？

著者：進化と勾配学習の共生は局所解の回避に効く。より深いネットへの適用が今後の課題。

所感：直接学習した全結合 AE（BCE 0.076）には届かず、比較相手は主に同パラメータ数の直接符号化。規模も MNIST にとどまる。

6 次に読むべき論文は？

- HyperNEAT (2007)（本資料）
- HyperNetworks (Ha+ 2016)（hypernetworks 分野で調査）
- CoDeepNEAT (2017)（本資料）
- 低ランク分解による間接符号化 (2025)（本資料）



出典：DPPN Fig. 2（DPPN が自己符号化器の重みを生成し、その勾配で DPPN を更新する二重ループ）

書誌情報

発表： GECCO 2016
リンク： https://arxiv.org/abs/1606.02580
系譜： 発展（← HyperNEAT / CPPN）
調査日： 2026-10-27
確認範囲： 全文（問題提起・手法・実験・結果・考察）

Evolving Deep Neural Networks

Risto Miikkulainen, Jason Liang, Elliot Meyerson ほか（Sentient Technologies / UT Austin）| arXiv プレプリント 2017

1 どんなもの？

NEAT のノードを「層」に置き換えた DeepNEAT と、モジュールと設計図の 2 集団を協調共進化させる CoDeepNEAT を提案。CIFAR-10 で誤差 7.3%（手設計並み）、PTB 言語モデルで LSTM 変種により perplexity を 5% 改善し、画像キャプション生成の実サービスにも使った。

2 先行研究と比べてどこがすごい？

NEAT は重みまで進化させるので小規模ネットに限られた。深層学習の構造設計は手作業か、CMA-ES によるハイパラ最適化や強化学習による NAS（Zoph & Le）に限られていた。重みは勾配で学び、構造・部品・ハイパラを進化で同時に最適化する二層最適化として定式化した。

3 技術や手法のキモはどこ？

DeepNEAT ではノードが層（種類・ユニット数・カーネル等）、辺が層間接続を表し、歴史的マーキングと種分化はそのまま使う。CoDeepNEAT では設計図の各ノードがモジュール種を指し、同じモジュールを繰り返し差し込んで網を組む。数エポック学習後の検証精度を適応度として両集団に配分する。

4 どうやって有効だと検証した？

CIFAR-10：毎世代 100 網を 8 エポック評価し 72 世代。最良網は誤差 7.3%（Snoek+ 6.4%）で、誤差 20% 到達は 12 エポック（同 30 超）。PTB で perplexity 78。キャプションは BLEU-4 29.1・CIDEr 88.0 で Show and Tell（27.7・85.5）を上回る（Table 3）。

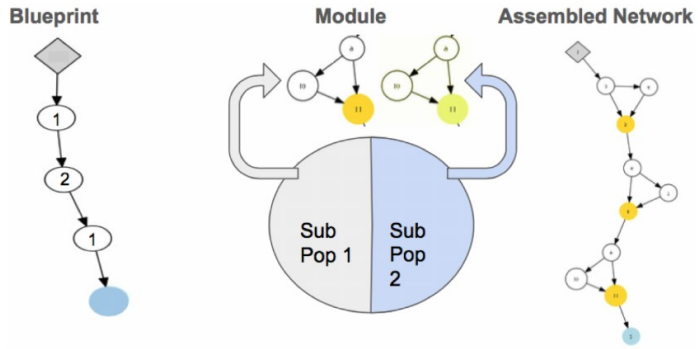
5 議論はある？

著者：計算資源が律速で、部分学習のため速く学習する網に偏る。資源が増えれば人手設計を超えると予測。

所感：約 100 GPU 規模の計算を要するが、ベースラインとの計算量比較がない。後の NAS（AmoebaNet など）と比べ精度の優位は示せていない。

6 次に読むべき論文は？

- Regularized Evolution / AmoebaNet (Real+ 2019)（未調査）
- EvoPrompting (Chen+ 2023)（未調査）
- LLMatic (Nasir+ 2024)（未調査）
- TensorNEAT (2024)（本資料）



出典：CoDeepNEAT Fig. 1（設計図の各ノードをモジュールで置き換えて網を組み立てる）

書誌情報

発表： arXiv プレプリント 2017（書籍章 2019）
リンク： https://arxiv.org/abs/1703.00548
系譜： 発展（← NEAT）
調査日： 2026-10-27
確認範囲： 全文（問題提起・手法・実験・結果・考察）

Weight Agnostic Neural Networks

Adam Gaier, David Ha（Bonn-Rhein-Sieg University of Applied Sciences / Google Brain）| NeurIPS 2019

1 どんなもの？

重みを学習せず、全結合に同じ 1 つの重みを入れるだけで課題を解くトポロジーを、NEAT 型の構造探索で見つける WANN を提案。CartPole 振り上げ・二足歩行・カーレースを無学習でこなし、MNIST でも重みのアンサンブルで正解率 91.6% を得た。

2 先行研究と比べてどこがすごい？

NAS が探すのは「訓練しやすい」構造で、解は重みの側にある。WANN は構造そのものに解（帰納バイアス）を持たせる。NEAT は構造と重みを同時に最適化するが、WANN は重みを捨てて構造変異だけを使い、重みを学習する内側ループを省いた。発想は MDL とベイズ NN に由来する。

3 技術や手法のキモはどこ？

最小構成から始め、ノード挿入・結合追加・活性化関数の変更（sin, ガウス, step など 10 種）で変異させる。各個体を共有重み $w \in \{-2,-1,-0.5,0.5,1,2\}$ で走らせ、平均性能・最大性能・結合数の 3 目的で非優越ソートする（8 割は平均性能と結合数で順位付け）。

4 どうやって有効だと検証した？

共有重みがランダムでも振り上げ 515（固定構造 7）、二足 51（同 −107）、カーレース 375（同 −85）。重みを個別調整すると 932・332・893 で既存手法並み（Table 1）。二足は 210 結合で既存手法の 1/13。MNIST はランダム重み 82.0%、調整 91.9%（Fig. 6）。

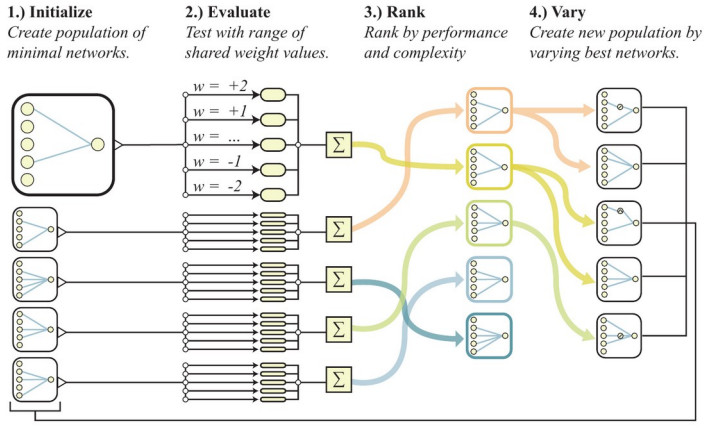
5 議論はある？

著者：CNN に及ばないのは当然で、新しい構成要素を見つけることが重要。生得的行動と学習の融合を目指す。

所感：重みを個別にランダムにすると失敗し、符号の一貫性に依存する。勾配での微調整は CMA-ES に劣り、多様な活性化関数が最適化地形を難しくする点は FER の観察とも通じる。

6 次に読むべき論文は？

- NEAT (2002)（本資料）
- Supermask (Zhou+ 2019)（未調査）
- FER 仮説 (2025)（本資料）
- Neural Developmental Programs (2023)（未調査）



出典：WANN Fig. 2（最小網から始め、共有重みを変えて評価し、性能と複雑さで順位付けして変異させる）

書誌情報

発表： NeurIPS 2019（spotlight）
リンク： https://arxiv.org/abs/1906.04358
系譜： 発展（← NEAT）
調査日： 2026-10-27
確認範囲： 全文（問題提起・手法・実験・結果・考察）

TensorNEAT: A GPU-accelerated Library for NeuroEvolution of Augmenting Topologies

Lishuang Wang, Mengfei Zhao, Enyu Liu ほか（ Southern University of Science and Technology ）| GECCO 2024 / ACM TELO 2025

1 どんなもの？

形の異なる NEAT 個体群を同じ形状のテンソルに変換する tensorization を提案し、JAX 上の GPU ライブラリ TensorNEAT を実装（ CPPN・HyperNEAT にも対応）。Brax のロボット制御で NEAT-Python 比最大 544 倍の高速化を達成した。

2 先行研究と比べてどこがすごい？

NEAT は個体ごとに構造が違うため GPU 並列化が難しく、EvoJAX や evosax など JAX 系の進化ライブラリから取り残されていた。既存の GPU 対応（ PyTorchNEAT ）は推論だけで、変異・交叉・距離計算・個体群評価は逐次。探索過程全体を個体群方向にベクトル化した。

3 技術や手法のキモはどこ？

各網をノード表と結合表に符号化し、上限 |N|max・|C|max まで NaN で埋めて個体群を 3 階テンソルに積む。ノード・結合の追加削除や属性の変更は行の書き換えに置き換え、推論はトポロジカル順と展開した結合テンソルで計算する。jax.vmap/pmap で全個体を一括処理。

4 どうやって有効だと検証した？

個体数 1 万・100 世代で、HalfCheetah は NEAT-Python（ CPU ）149,516 秒に対し RTX 4090 で 275 秒（ 544 倍）、Swimmer 196 倍、Hopper 60 倍（ Table 1 ）。TELO 版では evosax の GA 4 種と比べ、Hopper で NEAT 2354 対 SimpleGA 1294 など難課題で上回る（ Table 6 ）。

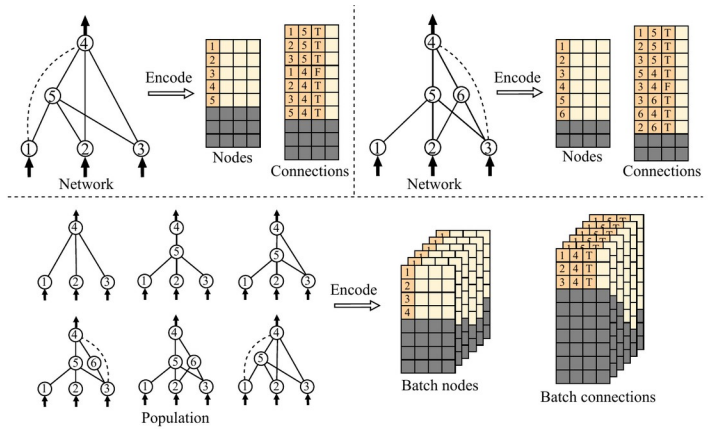
5 議論はある？

著者：分散環境への拡張と CoDeepNEAT 等の統合が課題。多 GPU は評価のみ並列で伸びが鈍る。

所感：ノード・結合数に固定上限が要り、NEAT の「際限ない複雑化」と緊張関係にある。速度差には OOP 実装の非効率も含む。

6 次に読むべき論文は？

- NEAT (2002)（本資料）
- evosax（並行テーマ neuroevolution-at-scale ）
- Quadtree Bottleneck (2026)（本資料）
- EvoX (Huang+ 2024)（未調査）



出典：TensorNEAT Fig. 2（構造の異なる網を NaN 埋めで同形状のノード・結合テンソルに符号化し個体群を束ねる）

書誌情報

発表： GECCO 2024 / ACM TELO 2025
リンク： https://arxiv.org/abs/2504.08339
系譜： 最新（← NEAT / HyperNEAT ）
調査日： 2026-10-27
確認範囲： 全文（ GECCO 2024 版と TELO 2025 拡張版の両方：手法・実験・考察）

Low Rank Factorizations are Indirect Encodings for Deep Neuroevolution

Jack Garbus, Jordan Pollack（Brandeis University）| arXiv プレプリント 2025

1 どんなもの？

重み行列を 2 つの低ランク因子の積で表し、因子だけを GA で変異させる低ランク分解型ニューロエボリューションを提案。Transformer 言語モデル（TinyStories）の全層を進化させ、非分解の直接符号化を大きく上回った。

2 先行研究と比べてどこがすごい？

Deep GA（Such+ 2017）は乱数シードで変異を圧縮でき大規模並列に強いが、直接符号化で探索空間が巨大。HyperNEAT などの間接符号化は実装も理解も難しい。LoRA が示す「解は低ランク」という深層学習の知見を、単純な間接符号化として進化に持ち込んだ。

3 技術や手法のキモはどこ？

m×n の重み W を A(m×k)·B(k×n) で「発生」させ、遺伝子型は k(m+n) 個の因子のみ（k=1 ～ 4、埋め込みは 32）。変異は因子にノイズを加え、変異率は平方根で補正。子は親 ID と乱数シードで表し $\theta_g = \theta_{g-1} + \sigma\,\varepsilon(\tau_g)$ として通信量を一定に保つ。

4 どうやって有効だと検証した？

TinyStories（7 試行）で分解版は同じ表現型サイズ・小型の非分解版より有意に良い（$p \ll 0.01$、Fig. 2）。小型非分解の遺伝子型は約 1.1 万で分解版の 7.6 万より小さく、探索空間の大きさでは説明できない。CarRacing は同等、Atari は Frostbite で有意に優位。悪い変異が早く脱落し世代時間が 38% 短縮。

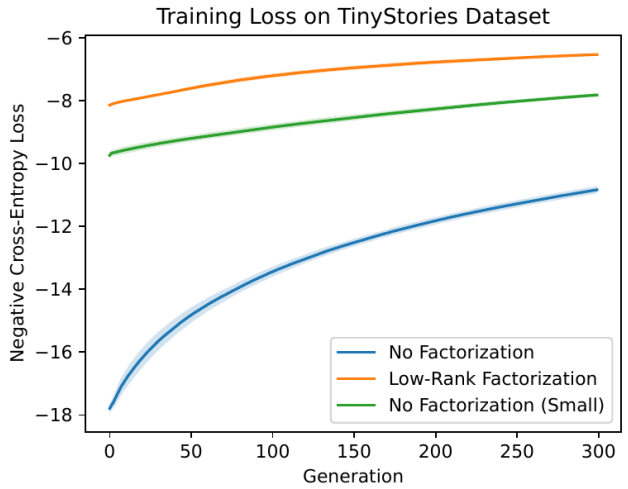
5 議論はある？

著者：埋め込み行列の分解が初期性能の主因。低ランクで探索した後に全ランク行列を足す併用も有望。

所感：言語モデルの損失は逆伝播の結果（2.38）に遠く及ばず、Atari はばらつきが大きい。トポロジーは固定で、NEAT 系とは別系統の間接符号化。

6 次に読むべき論文は？

- Deep GA (Such+ 2017)（並行テーマ neuroevolution-at-scale）
- LoRA (Hu+ 2021)（未調査）
- DPPN (2016)（本資料）
- Meta-evolving GENE (Kunze+ 2024)（未調査）



出典：低ランク分解 Fig. 2（TinyStories での最良個体の訓練損失。分解版が非分解版を上回る）

書誌情報

発表： arXiv プレプリント 2025
リンク： https://arxiv.org/abs/2504.03037
系譜： 最新（← DPPN / 間接符号化）
調査日： 2026-10-27
確認範囲： 全文（問題提起・手法・実験・結果・考察）

Questioning Representational Optimism in Deep Learning: The Fractured Entangled Representation Hypothesis

Akarsh Kumar, Jeff Clune, Joel Lehman, Kenneth O. Stanley（MIT ほか）| arXiv プレプリント 2025

1 どんなもの？

Picbreeder で開放的探索（NEAT＋人間の選択）によって得た CPPN と、同じ画像を SGD で学習させた CPPN を比較。出力は同一でも内部表現は大きく異なり、SGD 側は概念が分断・絡み合う「FER」を示すという仮説を提起した position paper。

2 先行研究と比べてどこがすごい？

「性能が上がれば表現も良くなる」という表現楽観論を問う。機構的解釈性は個々のニューロンに注目しがちで、網全体の表現戦略を評価する基準がなかった。CPPN は全ニューロンの挙動を画像で見られ、出力は同じで学習過程だけが違うという稀な比較対象になる。

3 技術や手法のキモはどこ？

Picbreeder の頭蓋骨 CPPN を等価な密 MLP に変換（層化）し、同じ構造をランダム初期化から SGD で同じ画像に回帰させる。各ニューロンの活性化を画像化し、重みを 1 本ずつ掃引して出力の変化を見る。対称性が早い層で一度だけ表現されるか（統一・因子化表現 UFR）を調べる。

4 どうやって有効だと検証した？

Picbreeder 版は早い層で左右対称が現れ、新しい中間表現は 24 個のみ（Fig. 5a）。重みの掃引で口の幅など意味のある変化が起きる（Fig. 6a）。SGD 版は最終層まで対称性を使わず雑多な模様が並び、掃引すると対称性が崩れる（Fig. 5b, 6b）。ReLU 網でも同様（付録 E）。

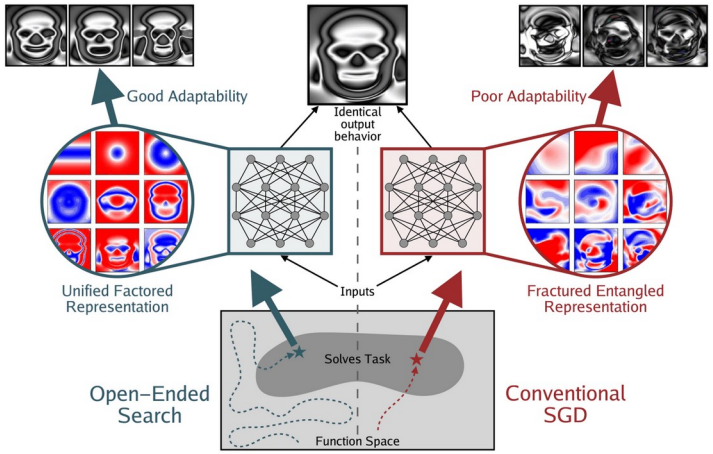
5 議論はある？

著者：FER は汎化・創造性・継続学習を損ないうる。学習の順序（対称→顔→頭蓋骨）と開放的探索が UFR の鍵と推測。

所感：比較は画像 1 枚の小さな設定で、LLM の事例は逸話的。定量指標がなく、仮説を検証する手段の整備が次の課題。

6 次に読むべき論文は？

- Picbreeder-VLM (2026)（既存：open-endedness/foundation-model-alife）
- Novelty Search (2011)（既存：open-endedness/quality-diversity）
- Picbreeder の進化可能性 (Huizinga+ 2018)（未調査）



出典：FER 仮説 Fig. 1（開放的探索と SGD は同じ出力に至るが、内部表現は UFR と FER に分かれる）

書誌情報

発表： arXiv プレプリント 2025（position paper）
リンク： https://arxiv.org/abs/2505.11581
系譜：最新（← CPPN / Picbreeder）
調査日： 2026-10-27
確認範囲：全文（本文と主要な付録：問題提起・比較実験・考察）

On Scaling Coordinate-Based Neuroevolution: The Quadtree Bottleneck in ES-HyperNEAT

Romain Claret, Michael O'Neill, Paul Cotofrei, Kilian Stoffel（ University of Neuchâtel ）| arXiv プレプリント 2026

1 どんなもの？

ES-HyperNEAT（ CPPN 出力の分散に応じた四分木で隠れノードの位置を自動発見）を JAX で GPU 実装した JAX-ESHN を作り、個体ごとに発見位置の数が違うため個体群をベクトル化できない「四分木ボトルネック」を構造的な限界として特定した。

2 先行研究と比べてどこがすごい？

TensorNEAT は NEAT と HyperNEAT を GPU 化したが、各 CPPN が基板の位置を適応的に決める ES-HyperNEAT は未対応。CPU 実装（ PUREPLES ）は深さとともに指数的に遅くなる。同グループのハイパラ探索・早期停止・入力分割は探索を安くするが、基板発見そのものは変えていない。

3 技術や手法のキモはどこ？

四分木は子 4 点の CPPN 出力の分散 $\sigma^2 > \tau$ なら分割を続ける（深さ d で最大 (4^(d+1)−1)/3 点）。この分岐が CPPN ごとに長さの違う位置集合を生み、静的形状を前提とする vmap/XLA と両立しない。CPPN 内のクエリー括化（ O1 ～ O4 ）や、固定格子で位置ごとに分散判定する HSHG で回避を試みた。

4 どうやって有効だと検証した？

CPPN 内の一括化は最大 1.73 倍で頭打ち（ Table 1 ）。HSHG は隠れノードを 29 ～ 63 個（四分木は 1 ～ 2 個）過剰に発見し、120 run すべて未解決。XOR 深さ 7 で CPU 版は解決率 4.2%・824 分、JAX-ESHN は 100%・348 分で、再試行込みで約 56 倍有利。ただし世代あたりは CPU 版が安い（約 27 分対 118 分）。

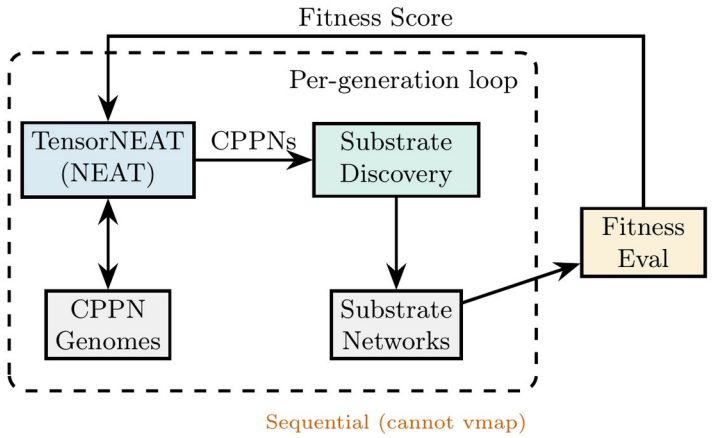
5 議論はある？

著者：適応性と静的形状でのベクトル化は本質的に対立する。共有の多解像度格子を一括評価し上からマスクする EMR-HyperNEAT が解決策（ XOR 深さ 7 で 34 倍）。

所感：課題は XOR などの玩具に限られ、ライブラリ差の交絡も残る。負の結果を丁寧に定量化した診断研究として価値がある。

6 次に読むべき論文は？

- ES-HyperNEAT (Risi & Stanley 2012)（本文未確認）
- EMR-HyperNEAT (Claret+ 2026)（未調査）
- Per-Node Activation Evolution (Claret+ 2026)（未調査）
- TensorNEAT (2024)（本資料）



出典：Quadtree Bottleneck Fig. 1（ TensorNEAT が CPPN を進化させ、基板発見は CPPN ごとに逐次実行される）

書誌情報

発表： arXiv プレプリント 2026
リンク： https://arxiv.org/abs/2608.24480
系譜： 最新（← ES-HyperNEAT / TensorNEAT ）
調査日： 2026-10-27
確認範囲： 全文（問題提起・手法・実験・結果・考察）